Web Application Development

Lecture 8: Context API

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain the **prop drilling problem** and why it is painful at scale
- Use the three-step Context API: `createContext` → **Provider** → `useContext`
- Wrap an application or subtree with a **Provider component**
- Read context values anywhere in the tree with `useContext`
- Build a **custom hook** that wraps `useContext` for cleaner APIs
- Implement a **ThemeContext** with toggle functionality
- Implement a **project-ready AuthContext** (login / logout / user)
- Compose **multiple contexts** and understand nesting

Agenda

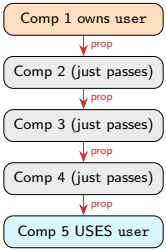
Outline

What is Prop Drilling?

W3Schools – React useContext Hook

React Context is a way to manage state globally. The component at the top and bottom of the stack need access to the state. To do this without Context, we will need to pass the state as "props" through each nested component. This is called **"prop drilling"**.

```jsx
// user lives in Component1
// Component5 needs it
// Components 2,3,4 just pass it through
// -- they don't use it at all!

function Component1() {
  const [user, setUser] = useState("Ahmed");
  return <Component2 user={user} />;
}
function Component2({ user }) {
  return <Component3 user={user} />;
}
function Component3({ user }) {
  return <Component4 user={user} />;
}
function Component4({ user }) {
  return <Component5 user={user} />;
}
function Component5({ user }) {
  // Finally! This is who needed it.
  return <h2>Hello {user}!</h2>;
}
```



Why Prop Drilling is Painful

Real-world consequences:

- Every intermediate component must **forward props it doesn't use**
- Adding a new consumer means **touching every layer** in between
- Rename a prop → **update every component** in the chain
- Makes component **APIs polluted** with irrelevant props
- Becomes a **maintenance nightmare** as the tree grows

Classic prop drilling scenario

Imagine a `user` object needed by a `UserAvatar` 8 levels deep – you'd have to pass it through 7 components that don't care about it at all.

W3Schools solution

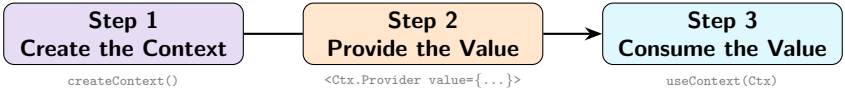
React Context is a way to manage state **globally**. It can be used together with `useState` to share state between deeply nested components **more easily** than with `useState` alone.

Common context use-cases:

- Logged-in user / auth state
- App theme (light / dark)
- Preferred language / locale
- Shopping cart
- App-wide settings

Outline

The Three-Step Context API



Step 1 – Create:

```
import { createContext }
  from 'react';

// Create the context object
// (usually in its own file)
const UserContext =
  createContext();

export default UserContext;
```

Step 2 – Provide:

```
import { useState } from 'react';
import UserContext from './
      UserContext';

function App() {
  const [user, setUser] =
    useState("Ahmed");

  return (
    <UserContext.Provider
      value={user}>
      <Component2 />
    </UserContext.Provider>
  );
}
```

Step 3 – Consume:

```
import { useContext } from 'react';
import UserContext from
  './UserContext';

function Component5() {
  // Direct access -- no props!
  const user =
    useContext(UserContext);

  return (
    <h2>Hello {user}!</h2>
  );
}
```

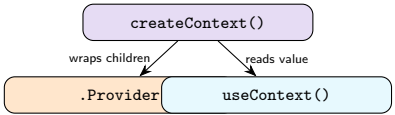
Step 1 – createContext

W3Schools – React useContext Hook

```
import { useState, createContext, useContext } from "react";
const UserContext = createContext();
```

```
import { createContext } from 'react';

// createContext() returns a Context object
// It has two key parts:
//   - UserContext.Provider  (parent side)
//   - consumed via useContext() (child side)
const UserContext = createContext();

export default UserContext;
```

```
// Optional: pass a default value
// Used ONLY when no Provider is above
const ThemeContext = createContext('light');
//                                 ^
//                          default fallback
```



Convention

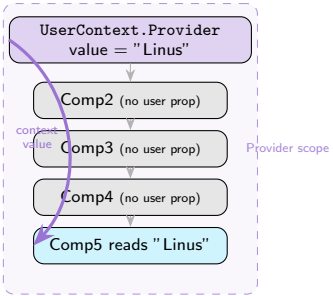
Create one context per concern, each in its own file: `ThemeContext.js`, `AuthContext.js`, etc.

Step 2 – The Provider

W3Schools – Context Provider

Next we use the Context Provider to **wrap the tree of components** that need the state Context. Wrap child components in the Context Provider and **supply the state value**.

```jsx
import { useState } from 'react';
import UserContext from './UserContext';

// W3Schools Component1 example
function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    // Wrap children in Provider
    // ALL descendants can read 'user'
    <UserContext.Provider value={user}>
      <h1>Hello {user}!</h1>
      <Component2 />
      {/* Component2 doesn't need user prop */}
    </UserContext.Provider>
  );
}
```



Step 3 – useContext

W3Schools – useContext Hook

In order to use the Context in a child component, we need to access it using the `useContext` Hook. First, include the useContext in the import statement. Then you can access the user Context in all components.

```jsx
// W3Schools Component3 example
import {
  useState,
  createContext,
  useContext
} from 'react';

const UserContext = createContext();

// Intermediate -- does NOT need user
function Component2() {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 />
    </>
  );
}

// Consumer -- reads context directly
function Component3() {
  const user = useContext(UserContext);

  return (
    <>
      <h1>Component 3</h1>
      <h2>Hello {user} again!</h2>
    </>
  );
}
```

Component 3 renders

Component 3

Hello **Linus** again!

What changed vs prop drilling:

- Component2 has **no user prop**
- Component3 reads context **directly**
- No props passed through intermediates
- Adding more consumers = **zero changes** to parents

useContext always finds nearest Provider

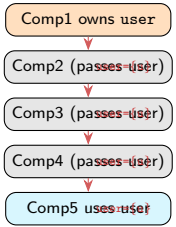
It walks **upward** in the tree and stops at the nearest matching Provider. No Provider found = uses `createContext` default value.

Full W3Schools useContext Example

```
import { useState, createContext, useContext } from 'react';
import { createRoot } from 'react-dom/client';

const UserContext = createContext();

function Component1() {
  const [user, setUser] = useState("Linus");
  return (
    <UserContext.Provider value={user}>
      <h1>Hello {user}!</h1>
      <Component2 />
    </UserContext.Provider>
  );
}

function Component2() {
  return ( <> <h1>Component 2</h1> <Component3 /> </> );
}
function Component3() {
  return ( <> <h1>Component 3</h1> <Component4 /> </> );
}
function Component4() {
  return ( <> <h1>Component 4</h1> <Component5 /> </> );
}

// Component5 is the only one that actually needs user
function Component5() {
  const user = useContext(UserContext);
  return (
    <>
      <h1>Component 5</h1>
      <h2>Hello {user} again!</h2>
    </>
  );
}

createRoot(document.getElementById('root')).render( <Component1 /> );
```

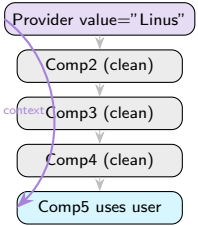
Prop Drilling vs Context – Visual Comparison

Without Context (prop drilling):



Problem: Comps 2, 3, 4 carry a prop they don't need.

With Context:



Solution: Comps 2, 3, 4 are clean. Comp5 reads directly.

Outline

Why Wrap useContext in a Custom Hook?

Without custom hook – verbose pattern

Every consumer must import *both* the context object and `useContext`, then call `useContext(SomeContext)`.

Without custom hook (repetitive):

```
// Every component that needs theme must:
import { useContext } from 'react';
import { ThemeContext } from '../ThemeContext';

function Navbar() {
  const { theme, toggleTheme } =
    useContext(ThemeContext);
  // ...
}

// And again in another component:
import { useContext } from 'react';
import { ThemeContext } from '../ThemeContext';

function Footer() {
  const { theme } = useContext(ThemeContext);
  // ...
}
```

With custom hook (clean):

```
// ThemeContext.js -- expose a hook
export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) {
    throw new Error(
      'useTheme must be used within ThemeProvider'
    );
  }
  return ctx;
}

// Now every consumer is just one import:
import { useTheme } from '../ThemeContext';

function Navbar() {
  const { theme, toggleTheme } = useTheme();
}

function Footer() {
  const { theme } = useTheme();
}
```

Custom Hook Pattern – Benefits

```
// ThemeContext.js -- complete file
import { createContext, useContext, useState }
  from 'react';

// 1. Create the context
const ThemeContext = createContext(null);

// 2. Provider component
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => {
    setTheme(t => t === 'light' ? 'dark' : 'light');
  };
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Custom hook -- wraps useContext
//    + helpful error if used outside Provider
export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error(
      'useTheme must be used within a ThemeProvider'
    );
  }
  return context;
}
```

Benefits of the custom hook:

- ✓ **Single import** per consumer (no context object import)
- ✓ **Helpful error** if used outside Provider
- ✓ Can add **logic** (validation, defaults) in one place
- ✓ **Encapsulates** the context implementation detail
- ✓ IDE provides **autocomplete** for returned values

Industry standard

This is the pattern used in **every real-world React project**. Libraries like React Router (`useNavigate`) and React Query (`useQuery`) follow exactly this pattern.

Outline

ThemeContext – Context File

```jsx
// src/context/ThemeContext.jsx
import { createContext, useContext, useState } from 'react';

// 1. Create
const ThemeContext = createContext(null);

// 2. Provider -- owns the state
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };

  // Provide BOTH value and the setter via an object
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Custom hook -- the only public API
export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within a ThemeProvider');
  }
  return context;
}
```

ThemeContext – App and Components

main.jsx – wrap the app:

```jsx
import { ThemeProvider } from
  './context/ThemeContext';
import App from './App';

createRoot(document.getElementById('root'))
  .render(
    <ThemeProvider>
      <App />
    </ThemeProvider>
  );
```

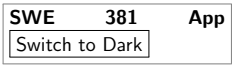
Navbar.jsx – reads theme:

```jsx
import { useTheme } from
  '../context/ThemeContext';

function Navbar() {
  const { theme, toggleTheme } = useTheme();
  return (
    <nav style={{
      background: theme === 'light'
        ? '#ffffff' : '#1a1a2e',
      color: theme === 'light'
        ? '#000' : '#fff'
    }}>
      <h1>SWE 381 App</h1>
      <button onClick={toggleTheme}>
        Switch to {theme === 'light'
          ? 'Dark' : 'Light'}
      </button>
    </nav>
  );
}
```

DeepButton.jsx – reads theme:

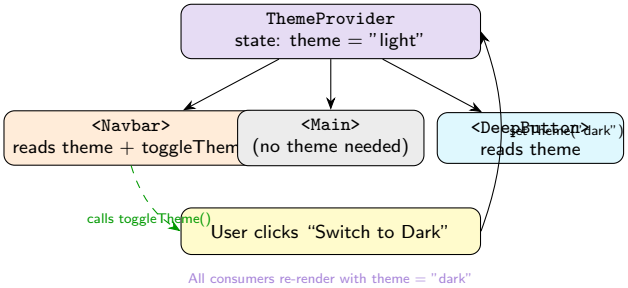
```jsx
// 8 levels deep -- no prop drilling!
import { useTheme } from
  '../context/ThemeContext';

function DeepButton({ label }) {
  const { theme } = useTheme();
  return (
    <button className={`btn-${theme}`}>
      {label}
    </button>
  );
}
```

Light theme



Dark theme

ThemeContext – How the Toggle Works



Outline

AuthContext – Why Authentication Needs Context

The auth problem without Context

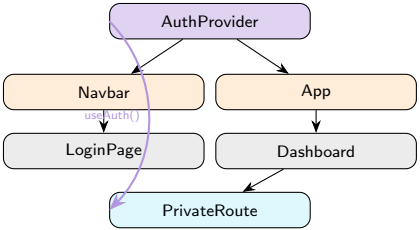
The logged-in user (`user` object) and auth actions (`login`/`logout`) are needed by the `Navbar`, the `Dashboard`, the `ProfilePage`, `PrivateRoute`, and many other components – all at different nesting levels. Prop drilling is impractical.

What AuthContext provides:

- `user` – the current logged-in user (or `null`)
- `login(userData)` – set the user
- `logout()` – clear the user
- `isAuthenticated` – derived boolean

Who needs it:

- `Navbar` – show user name / logout button
- `LoginPage` – call `login()`
- `Dashboard` – show personalised content
- `PrivateRoute` – redirect if not logged in



AuthContext – Context File

```jsx
// src/context/AuthContext.jsx
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

// Provider: owns user state + actions
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => {
    // In a real app: validate credentials with API first
    setUser(userData);
  };

  const logout = () => {
    setUser(null);
  };

  // Derived value -- no extra state
  const isAuthenticated = user !== null;

  return (
    <AuthContext.Provider value={{ user, login, logout, isAuthenticated }}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook with guard
export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
}
```

AuthContext – Consuming in Components

Navbar.jsx:

```jsx
import { useAuth } from '../context/AuthContext';

function Navbar() {
  const { user, logout, isAuthenticated }
    = useAuth();
  return (
    <nav>
      <h1>SWE 381</h1>
      {isAuthenticated ? (
        <>
          <span>Hi, {user.name}!</span>
          <button onClick={logout}>Log Out</button>
        </>
      ) : <span>Not logged in</span>}
    </nav>
  );
}
```

LoginPage.jsx:

```jsx
import { useAuth } from '../context/AuthContext';

function LoginPage() {
  const { login } = useAuth();
  const handleLogin = () => {
    login({ name: "Ahmed", role: "student" });
  };
  return (
    <button onClick={handleLogin}>
      Log In as Ahmed
    </button>
  );
}
```

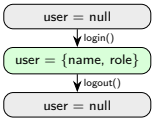
Not logged in

SWE 381 Not logged in

Log In as Ahmed

After login

SWE 381 Hi, Ahmed! Log Out



AuthContext – Protected Route Pattern

```jsx
// PrivateRoute.jsx
import { useAuth } from '../context/AuthContext';

function PrivateRoute({ children }) {
  const { isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return (
      <div>
        <h2>Access Denied</h2>
        <p>Please log in to view this page.</p>
      </div>
    );
  }

  return children;
}

// Usage in App.jsx
function App() {
  return (
    <AuthProvider>
      <Navbar />
      <PrivateRoute>
        <Dashboard />
      </PrivateRoute>
      <LoginPage />
    </AuthProvider>
  );
}
```

Not authenticated

Access Denied

Please log in to view this page.

Authenticated

Welcome to Dashboard!

Hello, Ahmed!
Role: student

Real-world extension

A production AuthContext also stores a JWT token in `localStorage`, checks expiry on mount with `useEffect`, and wraps the API call with the token in the header.

Outline

Using Multiple Contexts

Multiple contexts in one component

A component can call `useContext` (or use custom hooks) as many times as needed. Each context operates independently.

```jsx
// A component that needs BOTH contexts
import { useTheme } from '../context/ThemeContext';
import { useAuth }  from '../context/AuthContext';

function UserProfile() {
  // Each hook call is independent
  const { theme }                = useTheme();
  const { user, isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return <p>Please log in.</p>;
  }

  return (
    <div style={{
      background: theme === 'light' ? '#fff' : '#333',
      color:      theme === 'light' ? '#000' : '#fff',
      padding: '16px'
    }}>
      <h2>Welcome, {user.name}!</h2>
      <p>Current theme: {theme}</p>
      <p>Role: {user.role}</p>
    </div>
  );
}
```

Light + authenticated

Welcome, Ahmed!
Current theme: light
Role: student

Dark + authenticated

Welcome, Ahmed!
Current theme: dark
Role: student

Separate by concern

Keep each context focused on one area. Never create a single "god context" with everything – it causes every component to re-render whenever *anything* changes.

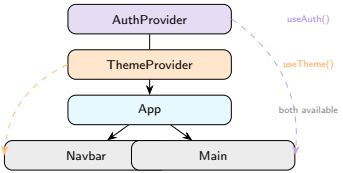
Nesting Providers – The AppProviders Pattern

React Docs – Nesting Providers

You can nest providers to supply different values to different parts of the tree. You can nest and override providers as many times as you need.

AppProviders.jsx – single file for all providers:

```jsx
import { AuthProvider }  from './context/AuthContext';
import { ThemeProvider } from './context/ThemeContext';

// Compose all providers in one place
function AppProviders({ children }) {
  return (
    <AuthProvider>
      <ThemeProvider>
        {children}
      </ThemeProvider>
    </AuthProvider>
  );
}

export default AppProviders;
```



main.jsx – clean entry point:

```jsx
import AppProviders from './AppProviders';
import App from './App';

createRoot(document.getElementById('root'))
  .render(
    <AppProviders>
      <App />
    </AppProviders>
  );
```

Context Nesting – Overriding Values in a Subtree

React Docs – Nested Providers

You can override the context for part of the tree by wrapping that part in a provider with a **different value**. The button inside the Footer receives a different context value ("light") than buttons outside ("dark").

```jsx
import { ThemeContext } from './context/ThemeContext';

// Top-level: dark theme for everything
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Navbar />          {/* gets "dark" */}
      <MainContent />     {/* gets "dark" */}

      {/* Override just the Footer subtree */}
      <ThemeContext.Provider value="light">
        <Footer />        {/* gets "light" -- overridden! */}
      </ThemeContext.Provider>

    </ThemeContext.Provider>
  );
}
// useContext always finds the NEAREST Provider above
```

Nearest Provider wins

useContext walks upward and stops at the **first** matching Provider it finds. This allows section-level theme overrides.

Use-case for nested Providers

Storybook component previews, admin sections with different colours, or wizard steps with isolated state all benefit from nested Providers.

When to Use Context – Decision Guide

Scenario	Use Context?	Reason
Data needed 1–2 levels deep	No	Props are fine
Data needed 3+ levels deep	Yes	Prop drilling gets painful
Multiple sibling components share data	Yes	Lift + context beats prop chain
Component is reusable library	No	Props keep it flexible
App-wide theme / language	Yes	Classic context use-case
Auth user across whole app	Yes	Classic context use-case
Data that changes every keystroke	No	Too many re-renders

Context re-renders all consumers

Every time the context `value` changes, **all** components calling `useContext` for that context re-render. Avoid putting fast-changing data (e.g., mouse position) in context.

Separate by update frequency

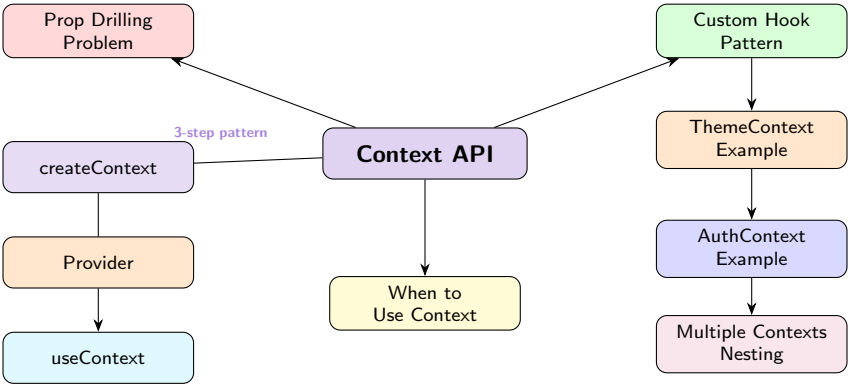
Split user info (rarely changes) and UI state (changes often) into **separate contexts** to minimise unnecessary re-renders.

Outline

Complete SWE381 App – All Contexts Together

```jsx
// main.jsx
import { AuthProvider }  from './context/AuthContext';
import { ThemeProvider } from './context/ThemeContext';

createRoot(document.getElementById('root')).render(
  <AuthProvider>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </AuthProvider>
);

// App.jsx
function App() {
  return (
    <>
      <Navbar />
      <PrivateRoute><Dashboard /></PrivateRoute>
      <LoginPage />
    </>
  );
}

// Dashboard.jsx -- uses BOTH contexts
import { useAuth }  from './context/AuthContext';
import { useTheme } from './context/ThemeContext';

function Dashboard() {
  const { user }  = useAuth();
  const { theme, toggleTheme } = useTheme();

  return (
    <main style={{
      background: theme === 'light' ? '#f8f9fa' : '#121212',
      color:      theme === 'light' ? '#212529' : '#e0e0e0',
      minHeight: '100vh', padding: '24px'
    }}>
      <h1>Welcome, {user?.name}!</h1>
      <p>Your dept: {user?.dept}</p>
      <button onClick={toggleTheme}>
        Switch to {theme === 'light' ? 'Dark' : 'Light'} mode
      </button>
    </main>
  );
}
```

Context API – Concept Map



File Structure for Context API

```
swe381-app/
  src/
    context/
      AuthContext.jsx    <- createContext
                            AuthProvider
                            useAuth()
      ThemeContext.jsx   <- createContext
                            ThemeProvider
                            useTheme()
    components/
      Navbar.jsx         <- import useAuth
                              useTheme
      Dashboard.jsx      <- import useAuth
                              useTheme
      PrivateRoute.jsx   <- import useAuth
      LoginPage.jsx      <- import useAuth
    App.jsx
    main.jsx             <- wrap with
    providers
```

What goes in each context file:

1. `createContext()` call
2. `Provider` component (owns state + actions)
3. `useXxx()` custom hook (with error guard)
4. Named exports for all three

One file per context

Keep all three pieces (context object, Provider, custom hook) in the same file. Consumers only need to import the custom hook.

Wrap in main.jsx

Put the outermost Provider in `main.jsx` or a dedicated `AppProviders.jsx` so your App component stays clean.

Outline

Lecture 8 – Key Takeaways

1. **Prop drilling** – passing props through components that don't need them – is a maintenance problem at scale
2. **React Context** solves this by making data available to any component in a subtree without explicit prop chains
3. **Three steps**: `createContext()` → `<Context.Provider value={...}>` → `useContext(Context)`
4. The **Provider** wraps the subtree that needs the data and supplies it via the `value` prop
5. `useContext` always reads from the **nearest matching Provider** above in the tree
6. Always wrap `useContext` in a **custom hook** (e.g. `useTheme()`) – centralises imports and adds error guards
7. **ThemeContext** – provides `theme` and `toggleTheme`; **AuthContext** – provides `user`, `login`, `logout`, `isAuthenticated`
8. Use **multiple contexts** (one per concern) rather than one giant context to avoid unnecessary re-renders
9. **AppProviders** pattern – nest all Providers in one file; wrap the entire app in `main.jsx`

Next Lecture: React Router – Client-Side Navigation & Dynamic Routes

Practice Exercises

1. Reproduce the **W3Schools useContext example** (Components 1–5). Verify that Component2, 3, and 4 do not receive a `user` prop, yet Component5 renders the user's name correctly.
2. Create a **ThemeContext** file with `ThemeProvider` and a `useTheme()` custom hook. Wrap the app with `ThemeProvider`. Add a toggle button in any deeply nested component that switches between light and dark.
3. Add the custom hook **error guard**: call `useTheme()` *outside* the `ThemeProvider` and confirm that the helpful error message is thrown. Then fix it by wrapping in the provider.
4. Build an **AuthContext** with `user` (null by default), `login(userData)`, and `logout()`. Add a `LoginPage` that calls `login()`, and a `Navbar` that shows the user name and a Logout button.
5. Implement a **PrivateRoute** component that uses `useAuth()` to protect a `Dashboard` page. Show "Access Denied" when not logged in and the dashboard content when logged in.
6. Combine both contexts in a **Dashboard** component that uses `useAuth()` and `useTheme()` simultaneously. Display the user name and allow theme switching.
7. **Challenge:** Create an **AppProviders.jsx** file that composes `AuthProvider` and `ThemeProvider` in one place. Use it in `main.jsx` to keep the entry point clean. Then add a third `LanguageContext` (EN/AR) and add a language toggle.